PRACTICAL – 1

1. Aim: Create different blinking patterns with LEDs by connecting to GPIO pins of Raspberry Pi.

2. Prior Concepts

- Concept of Python Programming
- Concept of Digital Electronics

3. New Concepts

- Introduction to Raspberry Pi 4
- Hardware interface of an LED with Raspberry Pi

4. Hardware Required

- Raspberry Pi 4
- LEDs (2 nos.)
- Breadboard
- Resistors (2 x 330Ω)
- Jumper wires

5. Procedure

1. Install Python3 IDLE on the Raspberry Pi: `sudo apt install idle3`
2. Connect the LEDs on the breadboard.
3. Connect a resistor to the positive end (anode) of each LED.
4. Connect the other end of each resistor to a GPIO pin on the Raspberry Pi.
5. Connect the negative end (cathode) of each LED to Ground.

6. Code

```
import RPi.GPIO as GPIO
from time import sleep

GPIO.setmode(GPIO.BOARD)

led1 = 29   # Yellow
led2 = 31   # Green

GPIO.setup(led1, GPIO.OUT)
GPIO.setup(led2, GPIO.OUT)
GPIO.output(led1, False)
GPIO.output(led2, False)


def ledpattern(v1, v2):
    GPIO.output(led1, v1)
    GPIO.output(led2, v2)

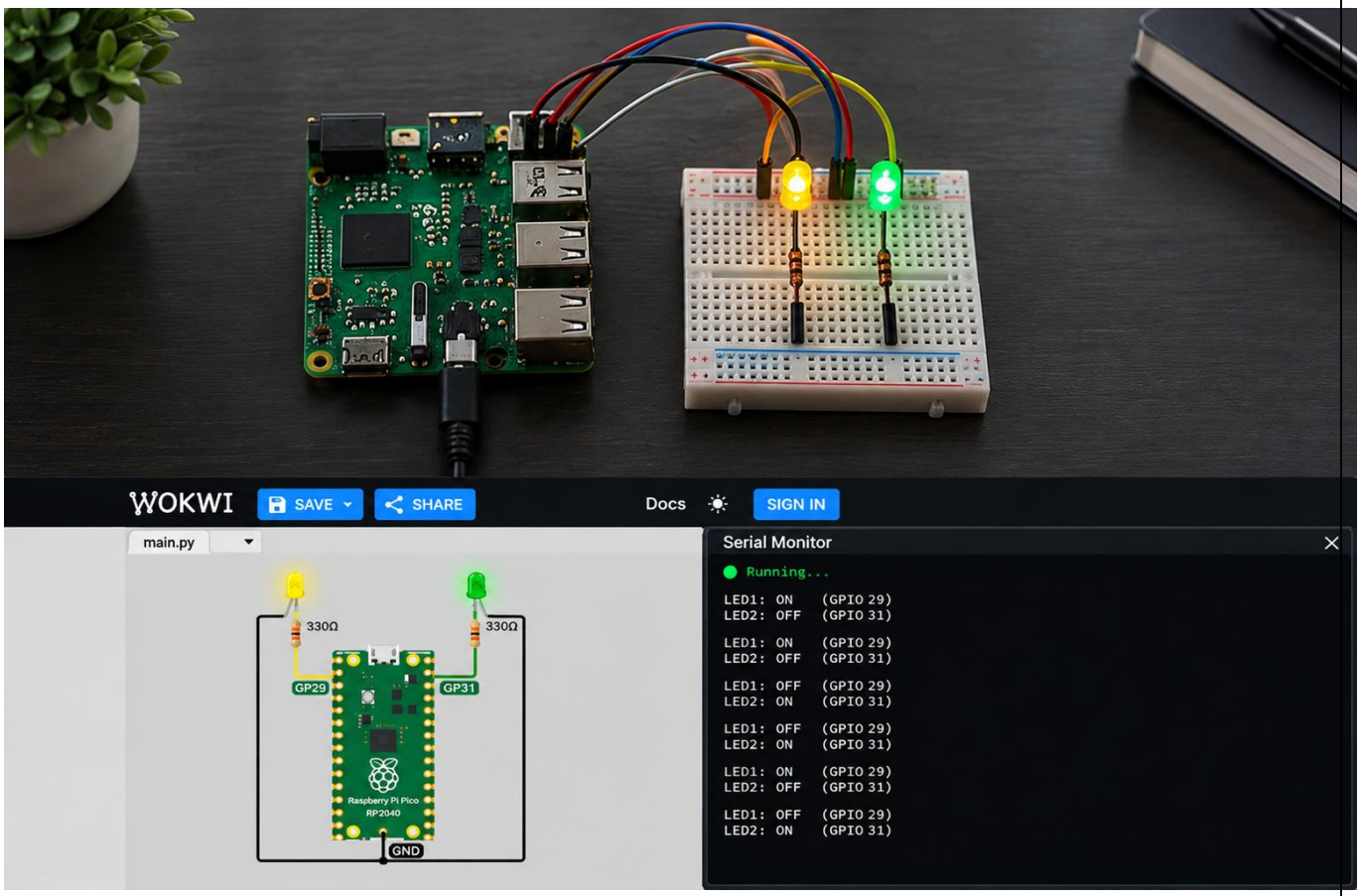

try:
    while True:
        ledpattern(1, 0)
        sleep(0.5)
        ledpattern(1, 1)
        sleep(0.5)
```

```
            ledpattern(0, 1)
            sleep(0.5)
            ledpattern(0, 0)
            sleep(0.5)
    finally:
        GPIO.output(led1,  False)
        GPIO.output(led2,  False)
        GPIO.cleanup()
```

Result / Observation:

The two LEDs cycle through a rotating binary pattern (10 → 11 → 01 → 00) with a 0.5 second gap between each state, confirming that both GPIO outputs switch correctly.

8. Output



PRACTICAL – 2

1. Aim: Use Telegram to control Raspberry Pi GPIO.

2. Prior Concepts

- Concept of Python Programming

3. New Concepts

- Controlling an LED through a Telegram bot and Raspberry Pi

4. Hardware Required

- Raspberry Pi 4
- LED
- Breadboard
- Resistor
- Jumper wires

5. Software Required – on Mobile Phone: Telegram

Steps to create a Telegram Bot

1. Download and install the Telegram app from the Play Store / App Store.
2. Register your mobile number with the Telegram service.
3. Open Telegram, search for "BotFather" and click Start.
4. Send `/newbot` to BotFather and follow the prompts to choose a name and a username (must end with "bot").
5. BotFather replies with an HTTP API access token – this token is used by your Python program to control the bot.

6. Procedure

1. Install Python3: `sudo apt install idle3`
2. Install the Telegram bot library: `pip3 install pyTelegramBotAPI` or `pip3 install telepot`
3. Replace `'Bot Token'` in the code below with the token received from BotFather.

7. Code

```
import sys
import time
import telepot
import RPi.GPIO as GPIO

GPIO.setmode(GPIO.BOARD)
GPIO.setup(11, GPIO.OUT)


def handle(msg):
    chat_id = msg['chat']['id']
    command = msg['text']
    print('Got command:', command)

    if command == 'on':
```

```
        GPIO.output(11, GPIO.HIGH)
        bot.sendMessage(chat_id, "LED on")
    elif command == 'off':
        GPIO.output(11, GPIO.LOW)
        bot.sendMessage(chat_id, "LED off")
    elif command == 'stop':
        exit()


bot = telepot.Bot('Bot Token')        # Replace with your own Bot Token
bot.message_loop(handle)
print('I am listening...')

try:
    while True:
        time.sleep(10)
except KeyboardInterrupt:
    GPIO.cleanup()
```

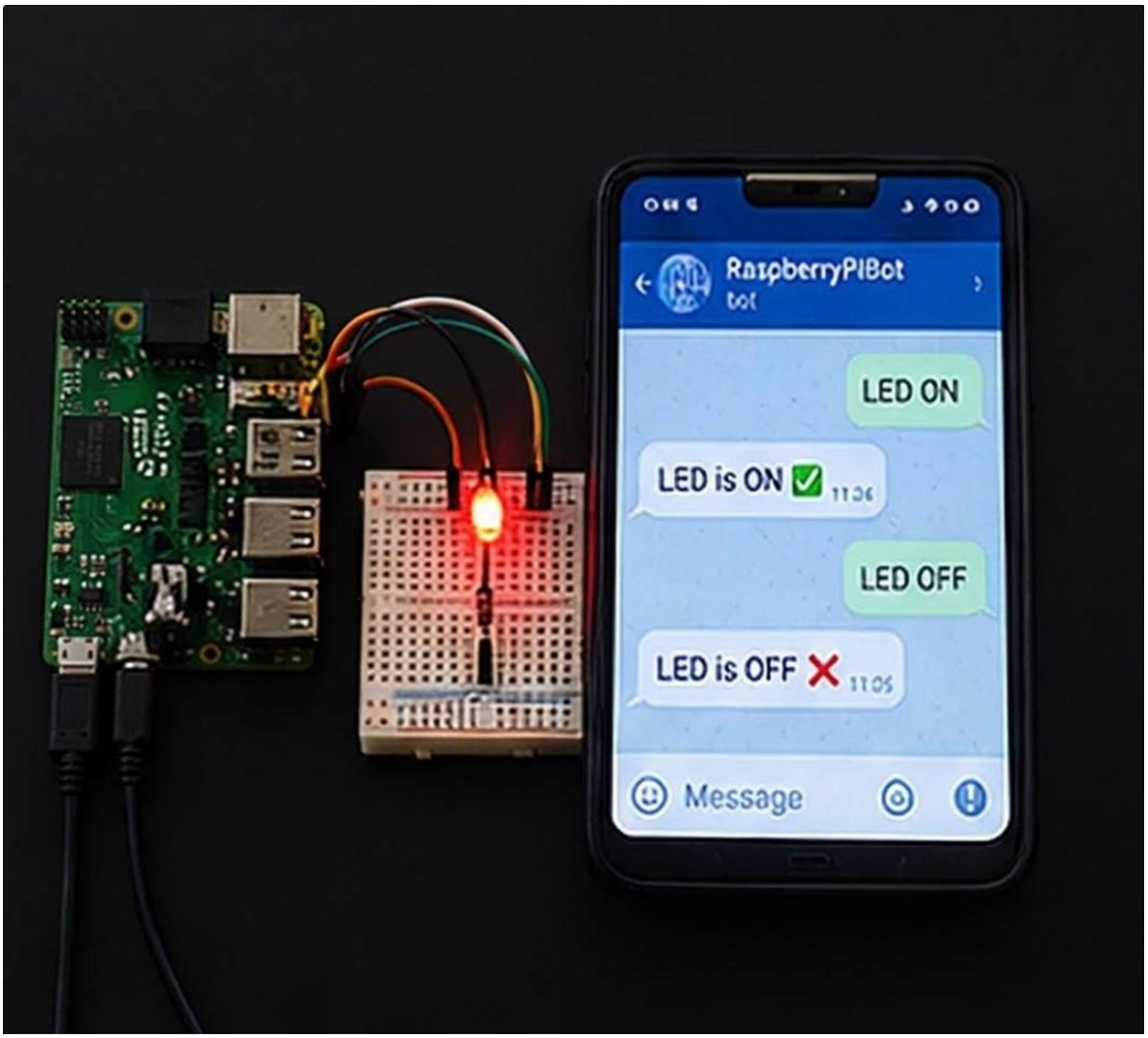
Note: Do not forget to replace 'Bot Token' with the token received from Telegram.

Result / Observation:

Sending “on” from the Telegram chat switches the LED ON, sending “off” switches it OFF, and sending “stop” terminates the program – each confirmed by a reply message with a timestamp in the chat.

8. Output

9. https://share.gemini.google/gSPix39CBe7t



PRACTICAL – 3

1. Aim: Interface TM1637 with Raspberry Pi and Display current time.

2. Prior Concepts

- Concept of Python Programming
- Concept of 7-Segment Displays

3. New Concepts

- Hardware connection of a 7-segment display with Raspberry Pi
- Displaying the current time on a 7-segment display

4. Hardware Required

- Raspberry Pi 4
- TM1637 4-digit 7-segment display board
- Jumper wires

Pin Connection

TM1637 Pin	Function	RPi Pin	Function
1	GND (Ground)	6	GND (Ground)
2	VCC (+5V Power)	2	5V
3	DIN (Data In)	38	GPIO 20
4	CLK (Clock)	40	GPIO 21

5. Procedure

1 Download the TM1637 driver library (`tm1637.py`) and save it in the same folder as the main program.

6. Code

main.py

```
import time
import tm163 as obj

# Create TM1637 display
Display = obj.TM1637(CLK=21, DIO=20, brightness=5)

# 7-segment codes for digits 0-9
digits = [
    0x3F,   # 0
    0x06,   # 1
    0x5B,   # 2
    0x4F,   # 3
    0x66,   # 4
    0x6D,   # 5
    0x7D,   # 6
    0x07,   # 7
    0x7F,   # 8
    0x6F    # 9
]

def display_time(hour, minute, colon=True):
    h1 = hour // 10
```

```
    h2 = hour % 10

    m1 = minute // 10
    m2 = minute % 10

    values = [
        digits[h1],
        digits[h2],
        digits[m1],
        digits[m2]
    ]

    # Blink colon
    if colon:
        values[1] |= 0x80

    Display.Show(values)

# Start time
hour = 12
minute = 37
second = 0

while True:

    display_time(hour, minute, second % 2 == 0)

    time.sleep(1)

    second += 1

    if second == 60:
        second = 0
        minute += 1

    if minute == 60:
        minute = 0
        hour += 1

    if hour == 24:
        hour = 0
```

diagram.json (Wokwi circuit)

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-pi-pico",
      "id": "pico",
      "top": -22.35,
      "left": 138,
      "attrs": { "env": "micropython-20260406-v1.28.0" }
    },
    {
      "type": "wokwi-tm1637-7segment",
      "id": "sevseg1",
      "top": 38.36,
      "left": -213.77,
      "attrs": { "color": "red" }
    }
  ],
  "connections": [
    [ "sevseg1:DIO", "pico:GP20", "green", [ "h0" ] ],
    [ "sevseg1:CLK", "pico:GP21", "green", [ "h0" ] ],
    [ "sevseg1:VCC", "pico:VBUS", "red", [ "h0" ] ],
    [ "sevseg1:GND", "pico:GND.4", "black", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

tm163.py (TM1637 driver library, imported by main.py)

```
from machine import Pin
import time

class TM1637:

    def __init__(self, CLK, DIO, brightness=5):
        self.clk = Pin(CLK, Pin.OUT)
        self.dio = Pin(DIO, Pin.OUT)
```

```
        self.clk.value(1)
        self.dio.value(1)

        self.brightness = brightness

        self.start()
        self.write_command(0x40)
        self.stop()

        self.clear()

    def start(self):
        self.dio.value(1)
        self.clk.value(1)
        time.sleep_us(2)
        self.dio.value(0)

    def stop(self):
        self.clk.value(0)
        time.sleep_us(2)
        self.dio.value(0)
        time.sleep_us(2)
        self.clk.value(1)
        time.sleep_us(2)
        self.dio.value(1)

    def write_byte(self, data):
        for i in range(8):

            self.clk.value(0)

            if data & 0x01:
                self.dio.value(1)
            else:
                self.dio.value(0)

            time.sleep_us(3)

            self.clk.value(1)
            time.sleep_us(3)

            data >>= 1

        self.clk.value(0)
        self.dio.value(1)
        time.sleep_us(2)
        self.clk.value(1)
        time.sleep_us(2)
        self.clk.value(0)

    def write_command(self, command):
        self.start()
        self.write_byte(command)
        self.stop()

    def brightness_set(self, brightness):
        if brightness < 0:
            brightness = 0

        if brightness > 7:
            brightness = 7

        self.brightness = brightness

    def clear(self):
        self.start()
        self.write_byte(0x40)
        self.stop()

        self.start()
        self.write_byte(0xC0)

        for i in range(4):
            self.write_byte(0x00)

        self.stop()

        self.start()
        self.write_byte(0x88 | self.brightness)
        self.stop()

    def show(self, data):

        self.start()
        self.write_byte(0x40)
        self.stop()
```

```
        self.start()
        self.write_byte(0xC0)

        for value in data:
            self.write_byte(value)

        self.stop()

        self.start()
        self.write_byte(0x88 | self.brightness)
        self.stop()

    def Show(self, data):
        self.show(data)

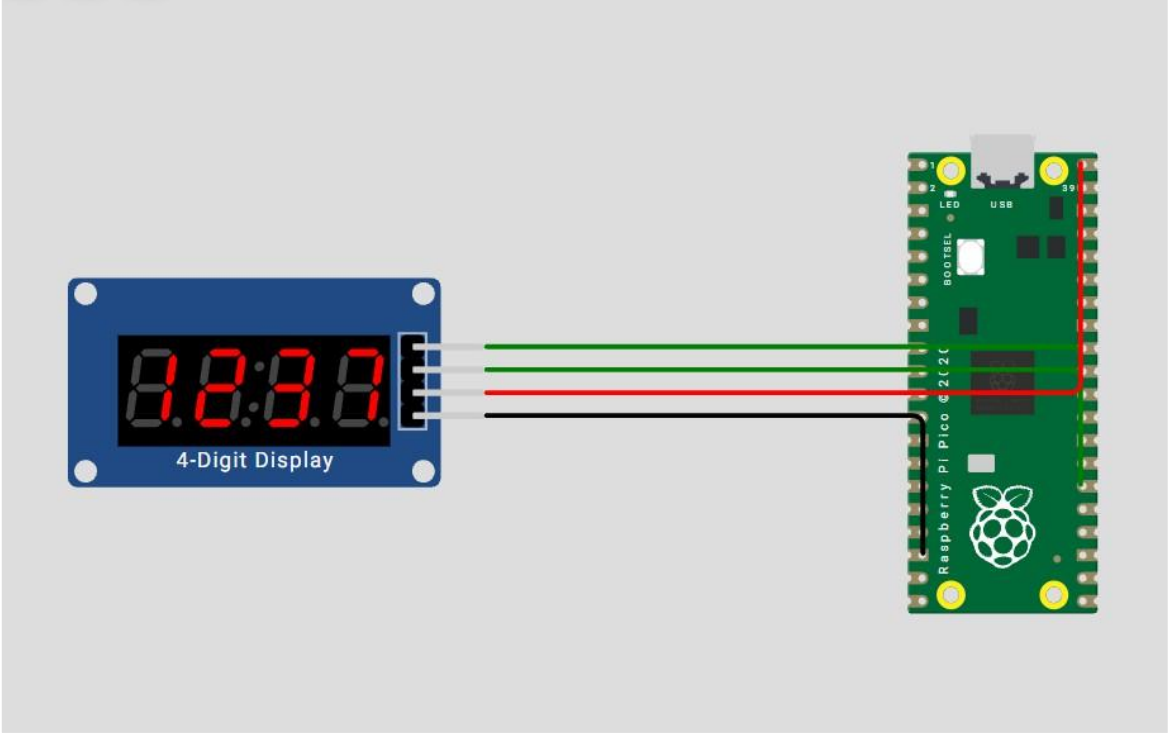
    def Clear(self):
        self.clear()

    def ShowDoublepoint(self, value):
        self.colon = value
```

Result / Observation:

The 4-digit display shows the time in HH:MM format (counting up from 12:37), with the centre colon blinking once every second.

8. Output



PRACTICAL – 4

1. Aim: Interface GPS module with Raspberry Pi.

2. Prior Concepts

- Concept of Python Programming
- Working of a GPS module sensor

3. New Concepts

- Hardware connection of a GPS module with Raspberry Pi

4. Hardware Required

- Raspberry Pi 4
- GPS Module (e.g. NEO-6M)
- Jumper wires

Pin Connection

GPS Module Pin	RPi Pin	Function
GND	6	GND (Ground)
VCC	2	5V
Tx	10	GPIO15 (RXD)
Rx	8	GPIO14 (TXD)

5. Procedure

1 Enable UART on Raspberry Pi: `sudo raspi-config` → Interface Options → Serial Port → "Login shell over serial?" → No, "Serial port hardware enabled?" → Yes.

2 Reboot: `sudo reboot`

3 Verify the serial port: `ls /dev/serial*` – expected output: `/dev/serial0`, `/dev/serial1`

4 Install GPS packages: `sudo apt update` then `sudo apt install gpsd gpsd-clients python3-gps -y`

5 Stop the GPS daemon so a custom script can access the port: `sudo systemctl stop gpsd.socket` and `sudo systemctl disable gpsd.socket`

6 Test the GPS data: `sudo gpsd /dev/serial0 -F /var/run/gpsd.sock` then `cgps -s` (or `gpsmon`). You should see NMEA sentences and GPS coordinates once the module acquires satellites, e.g. Time: 12:45:30 UTC, Latitude: 19.0760, Longitude: 72.8777, Altitude: 14.2 m.

7 Install `pip3 install pyserial pynmea2`

6. Code

```
import serial
import pynmea2

port = "/dev/serial0"
ser = serial.Serial(port, baudrate=9600, timeout=1)

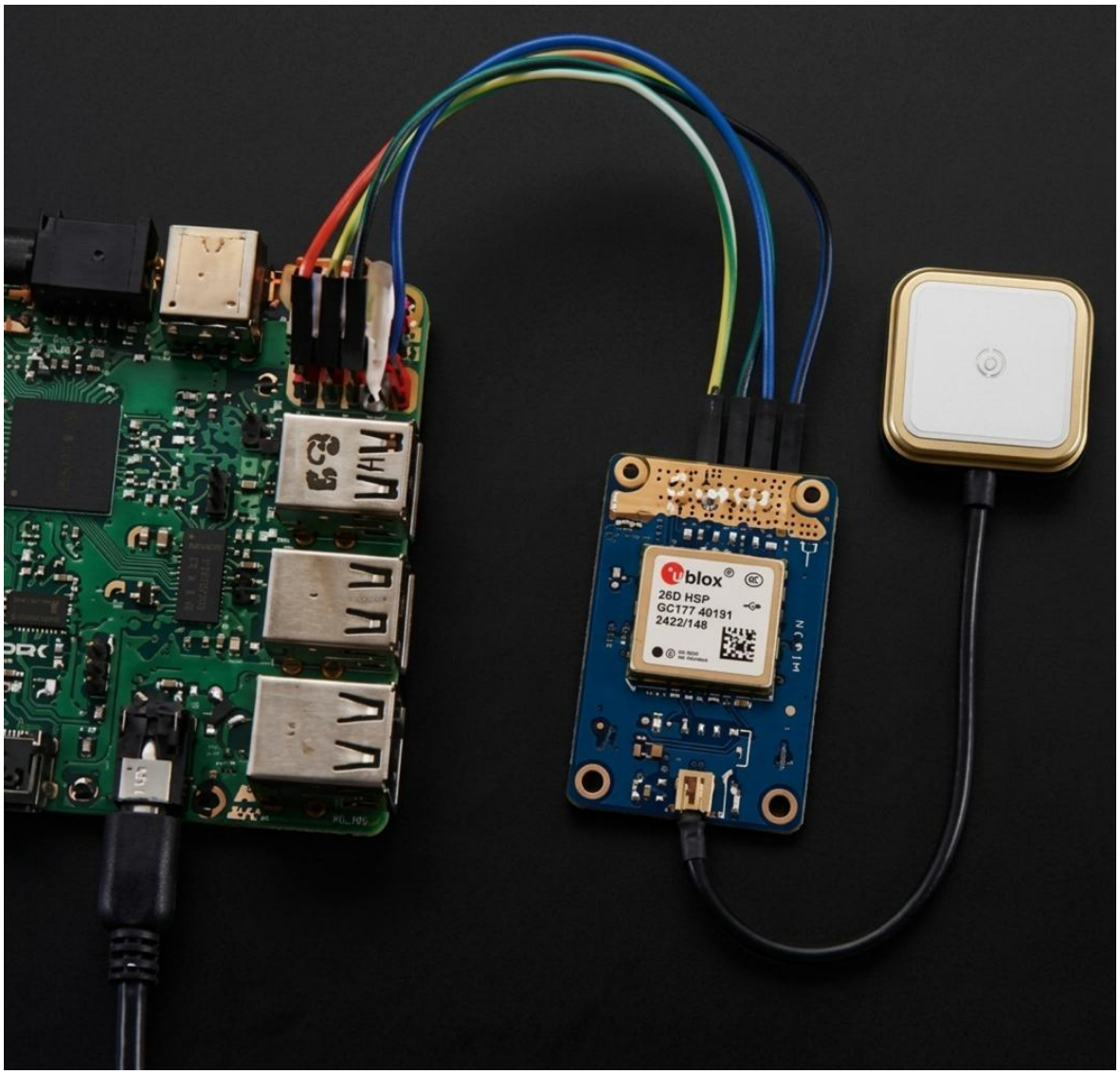
while True:
    data = ser.readline().decode('ascii', errors='re
```

```
if data.startswith('$GPGGA'):
    msg = pynmea2.parse(data)
    print("Latitude :", msg.latitude)
    print("Longitude:", msg.longitude)
```

Result / Observation:

Once the GPS module obtains a satellite fix, the program continuously prints the live latitude and longitude of the Raspberry Pi's location.

8. Output



PRACTICAL – 5

1. Aim: Interface ADS1115 with Raspberry Pi to act as an oscilloscope.

2. Prior Concepts

- Concept of Python Programming
- Use of a gas sensor
- Concept of an oscilloscope

3. New Concepts

- Hardware interfacing of a gas sensor with the ADS1115 ADC and Raspberry Pi
- Generating a live waveform of gas-sensor values

4. Hardware Required

- Raspberry Pi 4
- ADS1115 16-bit ADC module
- MQ-2 Gas Sensor
- Jumper wires

Pin Connection

ADS1115	RPi Pin	Function
VDD (5V)	2	5V
GND	6	GND (Ground)
SDA	3	GPIO 2 (SDA1)
SCL	5	GPIO 3 (SCL1)

MQ-2 Gas Sensor	ADS1115	RPi Pin	Function
VCC	–	4	5V
GND	–	39	GND (Ground)
A0	A0	–	–

Note: the MQ sensor output is 0–5V. Ensure the ADS1115 input does not exceed its VDD; use a voltage divider if needed.

5. Procedure

1. Enable the I2C interface: `sudo raspi-config` → Interface Options → I2C → Enable.
2. Update the system: `sudo apt update` and `sudo apt upgrade`
3. Install the Adafruit ADS1115 library and Matplotlib: `pip3 install adafruit-circuitpython-ads1x15 matplotlib --break-system-packages`
4. `pip3 uninstall -y adafruit-platformdetect adafruit-blinka`, then `sudo reboot`
5. `pip3 install --no-cache-dir --upgrade adafruit-platformdetect adafruit-blinka`, then `sudo reboot`
6. Install I2C tools and verify detection: `sudo apt-get install -y i2c-tools python3`, `lsmod | grep i2c_`, `i2cdetect -y 1`

6. Code

```
import time
import board
import busio
import matplotlib.pyplot as plt
from collections import deque
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn

i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1115(i2c)
ads.gain = 1  # +/- 4.096V

chan = AnalogIn(ads, ADS.P0)

x_vals = deque(maxlen=50)
y_vals = deque(maxlen=50)

plt.ion()
fig, ax = plt.subplots()
line, = ax.plot([], [], label="Gas Sensor Values from ADS1115", marker='o')
ax.set_ylim(0, 2)
ax.set_xlim(0, 100)
ax.set_title('MQ2 Gas Sensor Live Plot')
ax.set_xlabel('Sample')
ax.set_ylabel('Voltage (V)')
ax.legend()

counter = 0
print("Reading Gas Sensor values in Volts...!")

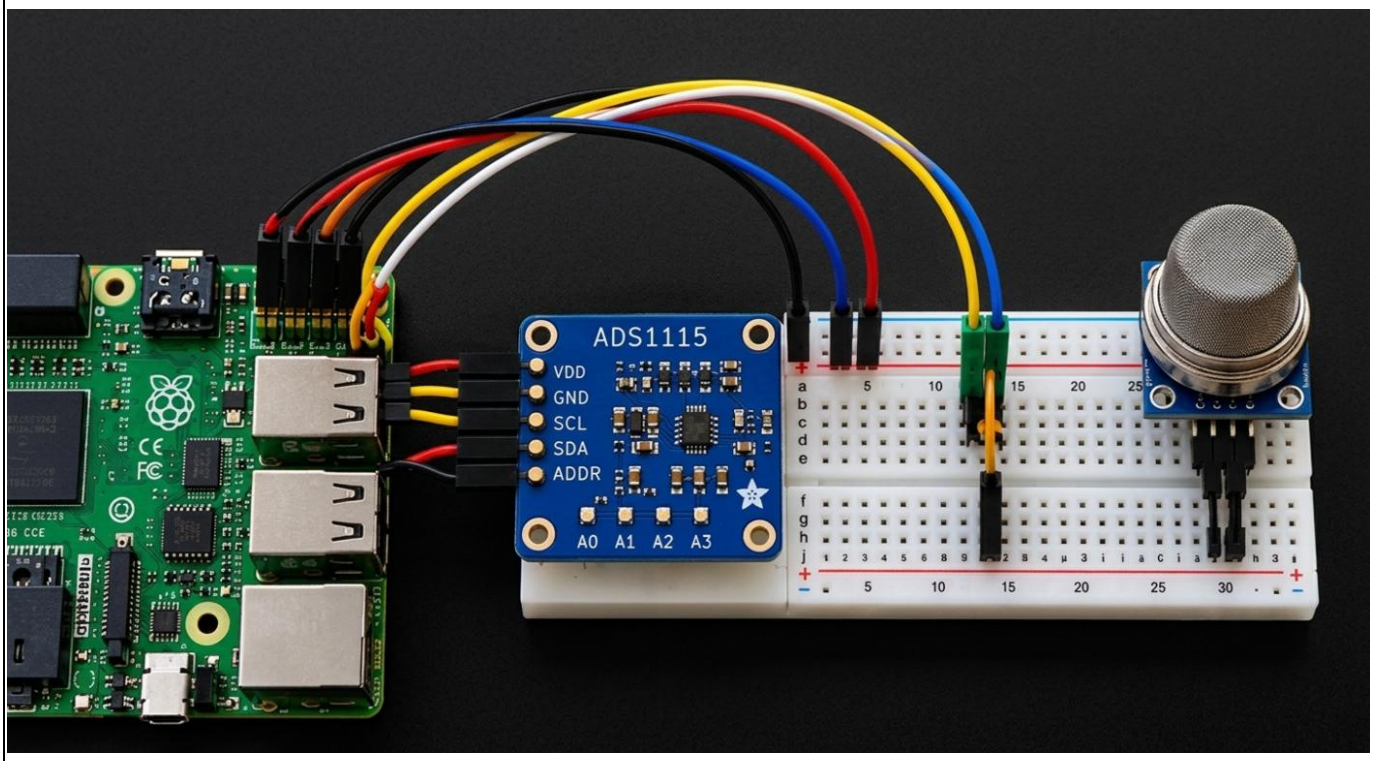
try:
    while True:
        v = chan.voltage
        x_vals.append(counter)
        y_vals.append(v)
        line.set_xdata(x_vals)
        line.set_ydata(y_vals)
        ax.set_xlim(counter - 100 if counter > 100 else 0, counter + 1)
        fig.canvas.draw()
        fig.canvas.flush_events()
        time.sleep(1)
        counter += 1
except KeyboardInterrupt:
    print("Stopped")
```

Result / Observation:

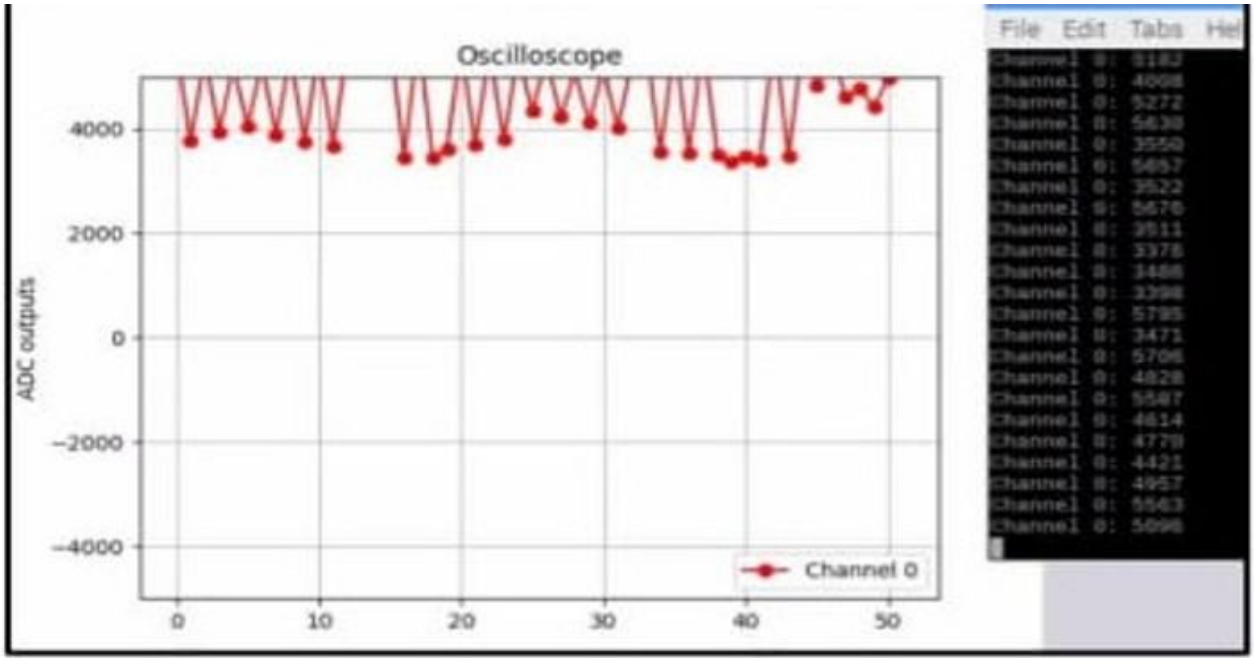
A live, oscilloscope-style graph is displayed showing the MQ-2 gas sensor's analog output voltage changing in real time, one sample per second.

8. Output

8. Output



Output 1: ADS1115 ADC and MQ-2 Gas Sensor interfaced with Raspberry Pi.



Output 2: Live oscilloscope-style ADC waveform from the MQ-2 gas sensor.

PRACTICAL – 6

1. Aim: Interface 16x2 LCD with Raspberry Pi using an I2C module to display text.

2. Prior Concepts

- Concept of Python Programming
- LCD function and use

3. New Concepts

- Hardware connection of a 16x2 LCD with Raspberry Pi using an I2C module

4. Hardware Required

- Raspberry Pi 4
- 16x2 LCD with I2C backpack (PCF8574)
- Jumper wires

Pin Connection

LCD I2C Module Pin	RPi Pin	Function
GND	9	GND (Ground)
VCC	2	5V
SDA	3	GPIO 2 (SDA1)
SCL	5	GPIO 3 (SCL1)

5. Procedure

1 Enable the I2C interface: `sudo raspi-config` → Interface Options → I2C → Enable, then `sudo reboot`

2 Install I2C tools: `sudo apt update`, `sudo apt install -y i2c-tools python3-smbus`

3 Verify the LCD is detected: `sudo i2cdetect -y 1` – expected address is typically `0x27` or `0x3F`.

4 Install the LCD library: `pip3 install RPLCD`

6. Code

main.py

```
from machine import I2C, Pin
from time import sleep

class SimpleLCD:
    def __init__(self, i2c, addr=0x27):
        self.i2c = i2c
        self.addr = addr
        self.backlight = 0x08
        for cmd in [0x33, 0x32, 0x28, 0x0C, 0x06, 0x01]:
            self._cmd(cmd)
        sleep(0.005)

    def _write(self, val):
        self.i2c.writeto(self.addr, bytes([val | self.backlight]))

    def _send(self, val, mode):
        high = (val & 0xF0) | mode
        low = ((val << 4) & 0xF0) | mode
        for b in [high | 0x04, high, low | 0x04, low]:
```

```
            self._write(b)

    def _cmd(self, cmd):
        self._send(cmd, 0)

    def clear(self):
        self._cmd(0x01)
        sleep(0.002)

    def move_to(self, col, row):
        addr = col + (0x40 if row else 0)
        self._cmd(0x80 | addr)

    def putstr(self, text):
        for char in text:
            if char == '\n':
                self.move_to(0, 1)
            else:
                self._send(ord(char), 1)

# Keep the exact I2C pin definition you used in your scanner script that succeeded
# If you used sda=Pin(0), scl=Pin(1), update the numbers below accordingly:
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
lcd = SimpleLCD(i2c, addr=0x27)

lcd.clear()
lcd.putstr("hello boiiizz\nRaspberry Pi 4")

sleep(10)
lcd.clear()
```

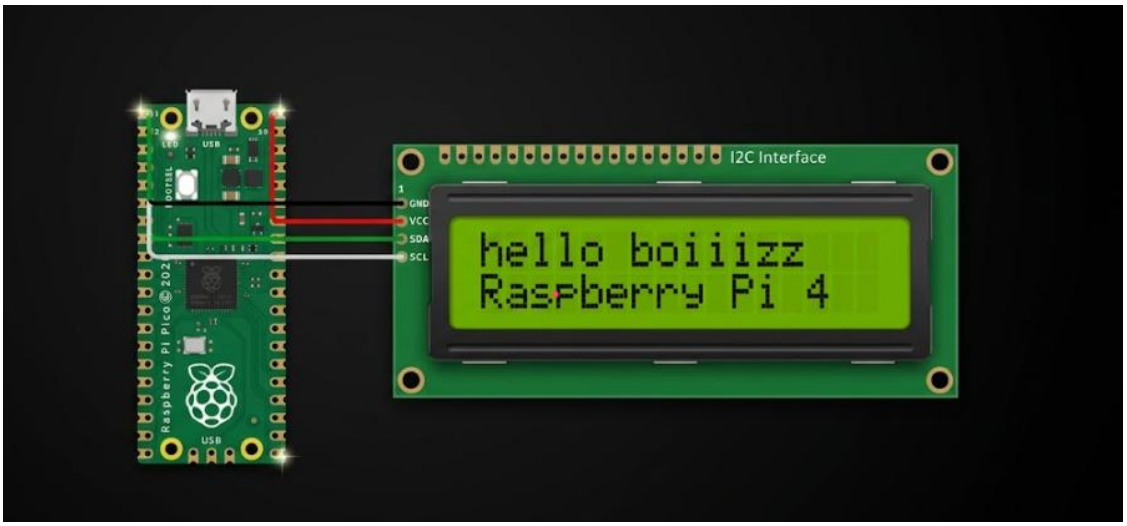
diagram.json (Wokwi circuit)

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-pi-pico",
      "id": "pico",
      "top": -108.75,
      "left": -159.6,
      "attrs": { "env": "micropython-20260406-v1.28.0" }
    },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": -80,
      "left": -23.2,
      "attrs": { "pins": "i2c" }
    }
  ],
  "connections": [
    [ "lcd1:VCC", "pico:VBUS", "red", [ "h0" ] ],
    [ "lcd1:GND", "pico:GND.1", "black", [ "h0" ] ],
    [ "lcd1:SDA", "pico:GP0", "green", [ "h0" ] ],
    [ "lcd1:SCL", "pico:GP1", "green", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

Result / Observation:

The LCD displays “hello boiiizz” on the first line and “Raspberry Pi 4” on the second line for 10 seconds, then clears.

8. Output



9.

PRACTICAL – 7

1. Aim: Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.

2. Prior Concepts

- Concept of Python Programming
- SPI communication interface

3. New Concepts

- Interfacing an RFID module with Raspberry Pi

4. Hardware Required

- RFID RC522 Reader Module
- A few RFID cards or tags
- Jumper wires

Pin Connection

RFID Module Pin	RPi Pin	Function
3.3V	1	3.3V
RST	22	GPIO 25
GND	6	GND (Ground)
MISO	21	GPIO 9 (SPI0 MISO)
MOSI	19	GPIO 10 (SPI0 MOSI)
SCK	23	GPIO 11 (SPI0 SCLK)
SDA	24	GPIO 8 (SPI0 CS0)

5. Procedure

1 Install Python3: `sudo apt install idle3`

2 Enable SPI for serial communication: `sudo raspi-config` → Interfacing Options → SPI → Yes, then reboot the Raspberry Pi.

3 `sudo apt update`

4 `pip3 install spidev`

5 `pip install mfrc522`

6. Code

main.py

```
from machine import Pin
from time import sleep

from mfrc522 import MFRC522

# ----------------------------------------
# RFID PIN CONFIGURATION
# ----------------------------------------

reader = MFRC522(
    sck=2,
    mosi=3,
    miso=4,
    rst=6,
```

```
    cs=5
)

# MIFARE Classic default key
KEY = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF]

# Data block
BLOCK = 4

# Sector trailer for block 4
TRAILER_BLOCK = 7

def get_card():

    print()
    print("Place RFID card on reader...")

    while True:

        status, tag_type = reader.request(
            reader.REQIDL
        )

        if status == reader.OK:

            status, uid = reader.anticoll()

            if status == reader.OK:

                reader.select_tag(uid)

                print("Card detected!")
                print(
                    "UID:",
                    ":".join(
                        "{:02X}".format(x)
                        for x in uid[:4]
                    )
                )

                return uid

        sleep(0.1)

def authenticate(uid):

    status = reader.auth(
        reader.AUTHENT1A,
        TRAILER_BLOCK,
        KEY,
        uid
    )

    return status == reader.OK

def write_tag():

    text = input(
        "Enter new data (maximum 16 characters): "
    )

    text = text[:16]

    uid = get_card()

    if not authenticate(uid):

        print("Authentication failed.")
        return

    data = list(text.encode())

    while len(data) < 16:
        data.append(32)

    status = reader.write(
        BLOCK,
        data
    )

    reader.stop_crypto1()

    if status == reader.OK:
        print("Data written successfully!")
```

```
        print("Written data:", text)
    else:
        print("Writing failed.")

    sleep(1)

def read_tag():

    uid = get_card()

    if not authenticate(uid):

        print("Authentication failed.")
        return

    data = reader.read(BLOCK)

    reader.stop_crypto1()

    if data is None:

        print("Reading failed.")
        return

    text = bytes(data[:16]).decode(
        "utf-8",
         "ignore"
    ).rstrip(" ")

    print()
    print("RFID CARD DETAILS")
    print("----------------- ")

    print(
        "UID:",
        ":".join(
            "{:02X}".format(x)
            for x in uid[:4]
        )
    )

    print("Data:", text)

    sleep(1)

def delete_tag():

    uid = get_card()

    if not authenticate(uid):

        print("Authentication failed.")
        return

    empty_data = [0] * 16

    status = reader.write(
        BLOCK,
        empty_data
    )

    reader.stop_crypto1()

    if status == reader.OK:
        print("Data deleted successfully!")
    else:
        print("Delete operation failed.")

    sleep(1)

print()
print("==============================")
print("      RFID PRACTICAL - 7")
print("==============================")
print("MFRC522 RFID Reader Ready")
print()

while True:

    print()
    print("Choose your option")
    print("1. Write your Tag")
    print("2. Read your Tag")
    print("3. Delete the content")
    print("4. Exit")
```

```
    option = input("Enter your option: ")

    if option == "1":

        write_tag()

    elif option == "2":

        read_tag()

    elif option == "3":

        delete_tag()

    elif option == "4":

        print("Program terminated.")
        break

    else:

        print("Invalid option.")
```

diagram.json (Wokwi circuit)

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-pi-pico",
      "id": "pico",
      "top": -3.15,
      "left": -121.2,
      "attrs": { "env": "micropython-20260406-v1.28.0" }
    },
    { "type": "board-mfrc522", "id": "rfid1", "top": 6.95, "left": 22.64, "attrs": {} }
  ],
  "connections": [
    [ "rfid1:3.3V", "pico:3V3_EN", "green", [ "h0" ] ],
    [ "rfid1:RST", "pico:GP6", "green", [ "h0" ] ],
    [ "rfid1:GND", "pico:GND.8", "black", [ "h0" ] ],
    [ "rfid1:MISO", "pico:GP4", "green", [ "h0" ] ],
    [ "rfid1:MOSI", "pico:GP3", "green", [ "h0" ] ],
    [ "rfid1:SCK", "pico:GP2", "green", [ "h0" ] ],
    [ "rfid1:SDA", "pico:GP5", "green", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

mfrc522.py (MFRC522 driver library, imported by main.py)

```
from machine import Pin, SPI
from time import sleep_us

class MFRC522:

    OK = 0
    NOTAGERR = 1
    ERR = 2

    REQIDL = 0x26
    REQALL = 0x52

    AUTHENT1A = 0x60
    AUTHENT1B = 0x61

    def __init__(self, sck, mosi, miso, rst, cs):

        self.sck = Pin(sck, Pin.OUT)
        self.mosi = Pin(mosi, Pin.OUT)
        self.miso = Pin(miso)
        self.rst = Pin(rst, Pin.OUT)
        self.cs = Pin(cs, Pin.OUT)

        self.cs.value(1)
        self.rst.value(0)

        self.spi = SPI(
            0,
            baudrate=1000000,
            polarity=0,
            phase=0,
            sck=self.sck,
            mosi=self.mosi,
```

```
            miso=self.miso
        )

        self.rst.value(1)

        self.init()

    def _wreg(self, reg, val):

        self.cs.value(0)

        self.spi.write(
            bytes([
                (reg << 1) & 0x7E,
                val
            ])
        )

        self.cs.value(1)

    def _rreg(self, reg):

        self.cs.value(0)

        self.spi.write(
            bytes([
                ((reg << 1) & 0x7E) | 0x80
            ])
        )

        value = self.spi.read(1)[0]

        self.cs.value(1)

        return value

    def _sflags(self, reg, mask):

        self._wreg(
            reg,
            self._rreg(reg) | mask
        )

    def _cflags(self, reg, mask):

        self._wreg(
            reg,
            self._rreg(reg) & (~mask)
        )

    def _tocard(self, command, send):

        recv = []
        bits = 0

        self._wreg(0x02, 0x77 | 0x80)
        self._cflags(0x04, 0x80)
        self._sflags(0x0A, 0x80)
        self._wreg(0x01, 0x00)

        for value in send:
            self._wreg(0x09, value)

        self._wreg(0x01, command)

        if command == 0x0C:
            self._sflags(0x0D, 0x80)

        timeout = 2000

        while timeout > 0:

            irq = self._rreg(0x04)

            if irq & 0x30:
                break

            if irq & 0x01:
                break

            timeout -= 1
            sleep_us(50)

        self._cflags(0x0D, 0x80)

        if timeout <= 0:
            return self.ERR, recv, bits

        if self._rreg(0x06) & 0x1B:
            return self.ERR, recv, bits
```

```
    if command == 0x0C:

        count = self._rreg(0x0A)
        last_bits = self._rreg(0x0C) & 0x07

        if last_bits:
            bits = (count - 1) * 8 + last_bits
        else:
            bits = count * 8

        if count == 0:
            count = 1

        if count > 16:
            count = 16

        for _ in range(count):
            recv.append(self._rreg(0x09))

    return self.OK, recv, bits

def _crc(self, data):

    self._cflags(0x05, 0x04)
    self._sflags(0x0A, 0x80)

    for value in data:
        self._wreg(0x09, value)

    self._wreg(0x01, 0x03)

    timeout = 255

    while timeout > 0:

        value = self._rreg(0x05)

        if value & 0x04:
            break

        timeout -= 1

    return [
        self._rreg(0x22),
        self._rreg(0x21)
    ]

def reset(self):

    self._wreg(0x01, 0x0F)

def antenna_on(self):

    value = self._rreg(0x14)

    if (value & 0x03) != 0x03:
        self._wreg(0x14, value | 0x03)

def init(self):

    self.reset()

    self._wreg(0x2A, 0x8D)
    self._wreg(0x2B, 0x3E)
    self._wreg(0x2D, 30)
    self._wreg(0x2C, 0)
    self._wreg(0x15, 0x40)
    self._wreg(0x11, 0x3D)

    self.antenna_on()

def request(self, mode):

    self._wreg(0x0D, 0x07)

    status, recv, bits = self._tocard(
        0x0C,
        [mode]
    )

    if status == self.OK and bits == 16:
        return self.OK, bits

    return self.ERR, 0

def anticoll(self):

    self._wreg(0x0D, 0x00)

    status, recv, bits = self._tocard(
        0x0C,
```

```
            [0x93, 0x20]
        )

        if status != self.OK:
            return self.ERR, []

        if len(recv) != 5:
            return self.ERR, []

        check = 0

        for i in range(4):
            check ^= recv[i]

        if check != recv[4]:
            return self.ERR, []

        return self.OK, recv

    def select_tag(self, uid):

        data = [0x93, 0x70] + uid[:5]

        data += self._crc(data)

        status, recv, bits = self._tocard(
            0x0C,
            data
        )

        if status == self.OK and bits == 24:
            return self.OK

        return self.ERR

    def auth(self, mode, block, key, uid):

        return self._tocard(
            0x0E,
            [mode, block] + key + uid[:4]
        )[0]

    def read(self, block):

        data = [0x30, block]

        data += self._crc(data)

        status, recv, bits = self._tocard(
            0x0C,
            data
        )

        if status == self.OK:
            return recv

        return None

    def write(self, block, data):

        command = [0xA0, block]

        command += self._crc(command)

        status, recv, bits = self._tocard(
            0x0C,
            command
        )

        if status != self.OK:
            return self.ERR

        if bits != 4:
             return self.ERR

        if not recv or (recv[0] & 0x0F) != 0x0A:
            return self.ERR

        data = list(data[:16])

        while len(data) < 16:
            data.append(0)

        data += self._crc(data)

        status, recv, bits = self._tocard(
            0x0C,
            data
        )
```

```
        if status == self.OK and bits == 4:
            if recv and (recv[0] & 0x0F) == 0x0A:
                return self.OK

        return self.ERR

    def stop_crypto1(self):

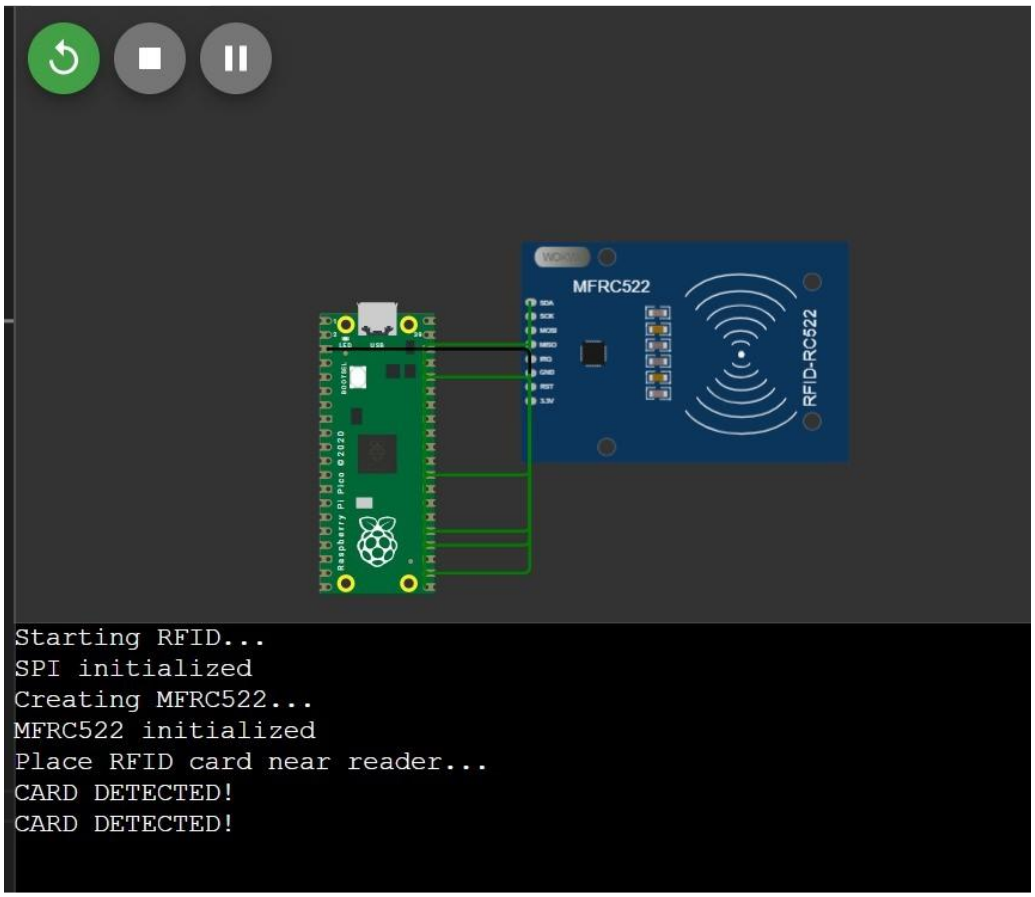
        self._cflags(0x08, 0x08)
```

Note: place the RFID cards one by one over the RFID module to check the output.

Result / Observation:

The program successfully writes text to an RFID tag, reads the tag's ID and stored text back, and clears (resets) the tag's data through the menu.

8. Output



PRACTICAL – 8

1. Aim: Interface Fingerprint Sensor with Raspberry Pi.

2. Prior Concepts

- Concept of Python Programming
- Serial (UART) communication interface

3. New Concepts

- Hardware connection of a fingerprint sensor with Raspberry Pi

4. Hardware Required

- Fingerprint Module (e.g. R307 / AS608)
- USB-to-TTL converter
- Jumper wires

5. Procedure

1. Install the fingerprint library: `pip install pyfingerprint`
2. Connect the fingerprint module to the Raspberry Pi through the USB-to-TTL converter (TXD → RXD, RXD → TXD, VCC → 5V, GND → GND).

6. Code

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

import time
import RPi.GPIO as GPIO
from pyfingerprint.pyfingerprint import PyFingerprint

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.cleanup()

# ---------------- Initialize Fingerprint Module ----------------
try:
    fingerprint_module = PyFingerprint('/dev/ttyUSB0', 57600, 0xFFFFFFFF, 0x00000000)
    if not fingerprint_module.verifyPassword():
        raise ValueError('The given fingerprint sensor password is wrong!')
except Exception as e:
    print('Exception message: ' + str(e))
    exit(1)


def delaysec(t):
    time.sleep(t)


def enrollFingerinDB():
    print("Enroll your Finger into Fingerprint Database")
    try:
        print('Keep your finger...')
        while fingerprint_module.readImage() == False:
            pass
        fingerprint_module.convertImage(0x01)
```

```
        print('Remove your finger and place it again...')
        while fingerprint_module.readImage() == False:
            pass
        fingerprint_module.convertImage(0x02)

        if fingerprint_module.compareCharacteristics() == 0:
            raise Exception('Fingers do not match')

        fingerprint_module.createTemplate()
        positionNumber = fingerprint_module.storeTemplate()
        print('Finger enrolled successfully!')
        print('New template position # ' + str(positionNumber))
    except Exception as e:
        print('Enrollment fail! Error: ' + str(e))


def searchFingerinDB():
    print("Search your Finger into Fingerprint Database")
    try:
        print('Keep your finger...')
        while fingerprint_module.readImage() == False:
            pass
        fingerprint_module.convertImage(0x01)

        result = fingerprint_module.searchTemplate()
        positionNumber = result[0]

        if positionNumber == -1:
            print('No fingerprint template match found.')
        else:
            print('Fingerprint found at position number = ' + str(positionNumber))
    except Exception as e:
        print('Searching fail! Error: ' + str(e))


def deleteFingerinDB():
    print("Delete your Finger data from Fingerprint Database")
    try:
        positionNumber = int(input('Please enter the template position to delete (0-1000): '))
        if fingerprint_module.deleteTemplate(positionNumber) == True:
            print('Template deleted successfully.')
    except Exception as e:
        print('Deletion fail! Error: ' + str(e))


def getFingerprintTemplateCount():
    template = fingerprint_module.getTemplateCount()
    print('Fingerprint template count in database = ' + str(template))
    print('Fingerprint storage capacity = ' + str(fingerprint_module.getStorageCapacity()))


def main():
    print("Initialization done... Starting program")
    delaysec(1)

    while True:
        print("\n\nChoose an option:\n 1. Enroll Fingerprint\n 2. Search Fingerprint\n "
              "3. Delete Fingerprint\n 4. Get fingerprint record count\n 5. Exit Program\n")
        choice = int(input("Enter choice: "))

        if choice == 1:
            enrollFingerinDB()
        elif choice == 2:
            searchFingerinDB()
        elif choice == 3:
```

```
            deleteFingerinDB()
        elif choice == 4:
            getFingerprintTemplateCount()
        elif choice == 5:
            exit(0)

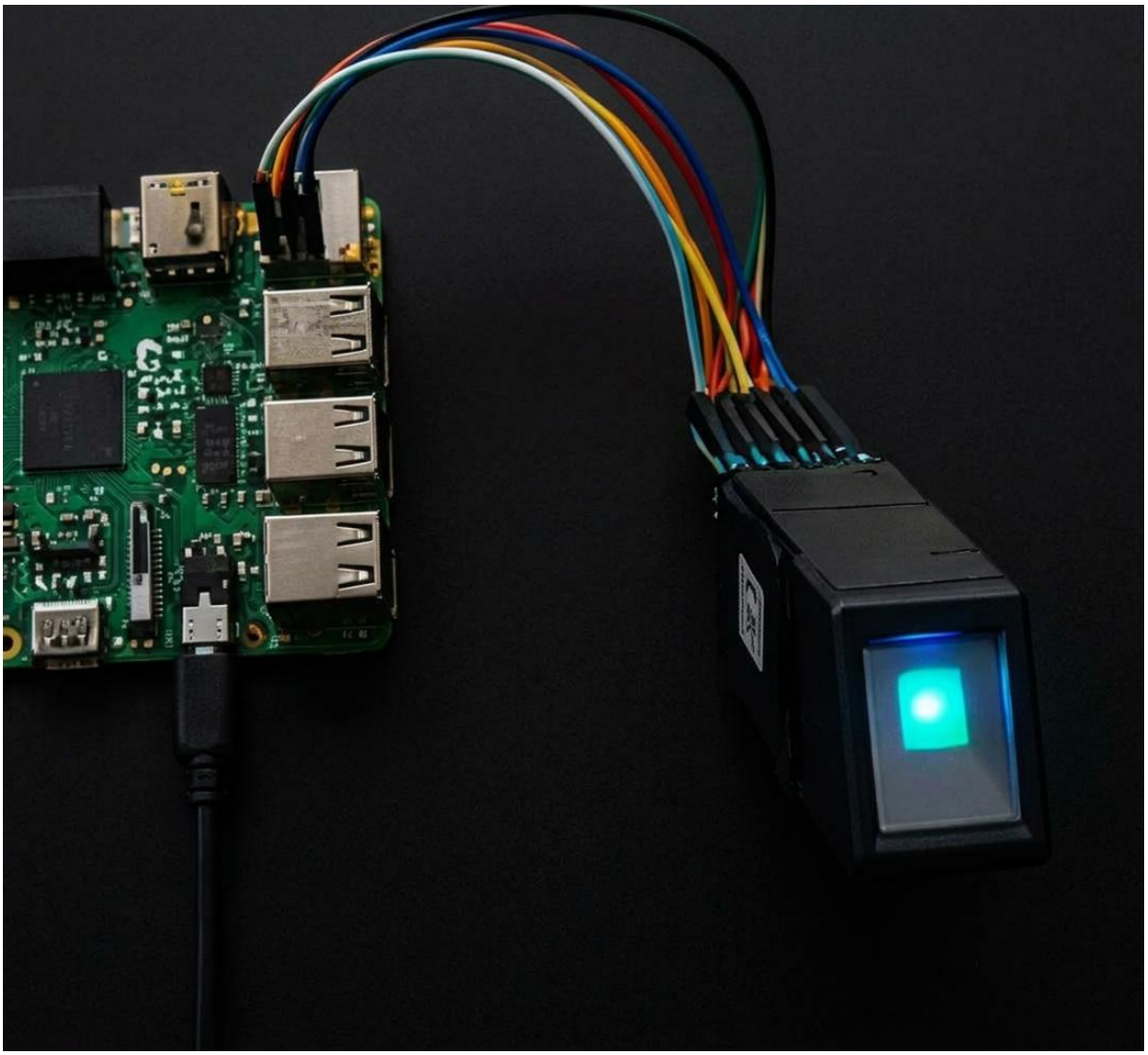
        delaysec(0.5)


if __name__== '__main__':
    try:
        main()
    except KeyboardInterrupt:
        pass
    finally:
        GPIO.cleanup()
        print("\n\nProgram terminated...")
```

Result / Observation:

The menu-driven program successfully enrolls a new fingerprint, searches for a stored match, deletes a template by position number, and reports the current template count in the sensor's database.

8. Output



PRACTICAL – 9

1. Aim: Interface Pi Camera with Raspberry Pi to capture image and video.

2. Prior Concepts

- Basics of Raspberry Pi and Linux OS
- Python programming fundamentals
- GPIO and peripheral interfacing concepts

3. New Concepts

- Working of the Pi Camera Module
- CSI (Camera Serial Interface) communication
- Capturing images using Python
- Recording videos using Python
- Using the picamera2 library
- Camera preview and media storage handling

4. Hardware Required

- Raspberry Pi Camera Module
- Ribbon / jumper cable

5. Procedure

1. Install Python3: `sudo apt install python3-idle3`
2. Connect the camera ribbon cable to the CSI Camera Port on the Raspberry Pi (blue side facing the USB ports), and press the clip down to lock it firmly.
3. `sudo apt update`
4. `sudo apt install python3-picamera2 -y`
5. `pip install opencv-python numpy`

6. Code

```
from picamera2 import Picamera2
from time import sleep

# Initialize camera
picam2 = Picamera2()

# Start camera
picam2.start()

# Allow camera to warm up
sleep(2)

# -------------- Capture Image --------------
picam2.capture_file("captured_image.jpg")
print("Image Captured Successfully")

# -------------- Record Video --------------
picam2.start_recording("captured_video.h264")
print("Video Recording Started")

sleep(10)    # record for 10 seconds
```

```
picam2.stop_recording()
print("Video Recorded Successfully")

picam2.stop()
print("Program Completed")
```

Result / Observation:

An image file “captured_image.jpg” and a 10-second video “captured_video.h264” are saved in the working directory.

8. Output



PRACTICAL – 10

1. Aim: Design an IoT based home automation system using Raspberry Pi.

2. Prior Concepts

- Concept of Python Programming
- Working of relay modules
- Basic networking concepts

3. New Concepts

- IoT-based remote appliance control
- Interfacing relay modules with Raspberry Pi
- Web-based appliance automation
- Flask web server programming
- GPIO control using Python
- Smart home automation concepts

4. Hardware Required

- 2-channel Relay Module
- Bulb
- DC Fan
- Jumper wires

Pin Connection

GPIO Pin (BCM)	Physical Pin	Description	Relay Module Pin
GPIO17	Pin 11	Control Light (AC)	IN1
GPIO27	Pin 13	Control DC Fan (12V)	IN2
5V	Pin 2 or 4	Power supply	VCC
GND	Pin 6	Ground	GND

Working: the Raspberry Pi connects to Wi-Fi, the Flask web server running on it accepts ON/OFF commands from a mobile/web browser, and the corresponding GPIO pin drives the relay that switches the connected appliance.

5. Procedure

1. Install Python3: `sudo apt install idle3`
2. `pip install flask`
3. `pip install RPi.GPIO`

6. Code

```
from flask import Flask
import RPi.GPIO as GPIO

app = Flask(__name__)

GPIO.setmode(GPIO.BCM)
```

```
LIGHT = 17
FAN = 27
GPIO.setup(LIGHT, GPIO.OUT)
GPIO.setup(FAN, GPIO.OUT)

GPIO.output(LIGHT, GPIO.LOW)
GPIO.output(FAN, GPIO.LOW)


@app.route('/')
def home():
    return '''
    <h1>IoT Home Automation</h1>
    <h3>Light</h3>
    <a href="/light/on">ON</a><br><br>
    <a href="/light/off">OFF</a><br><br>
    <h3>Fan</h3>
    <a href="/fan/on">ON</a><br><br>
    <a href="/fan/off">OFF</a>
    '''


@app.route('/light/on')
def light_on():
    GPIO.output(LIGHT, GPIO.HIGH)
    return "Light ON"


@app.route('/light/off')
def light_off():
    GPIO.output(LIGHT, GPIO.LOW)
    return "Light OFF"


@app.route('/fan/on')
def fan_on():
    GPIO.output(FAN, GPIO.HIGH)
    return "Fan ON"


@app.route('/fan/off')
def fan_off():
    GPIO.output(FAN, GPIO.LOW)
    return "Fan OFF"


if __name__== '__main__':
    app.run(host='0.0.0.0', port=5000)
```

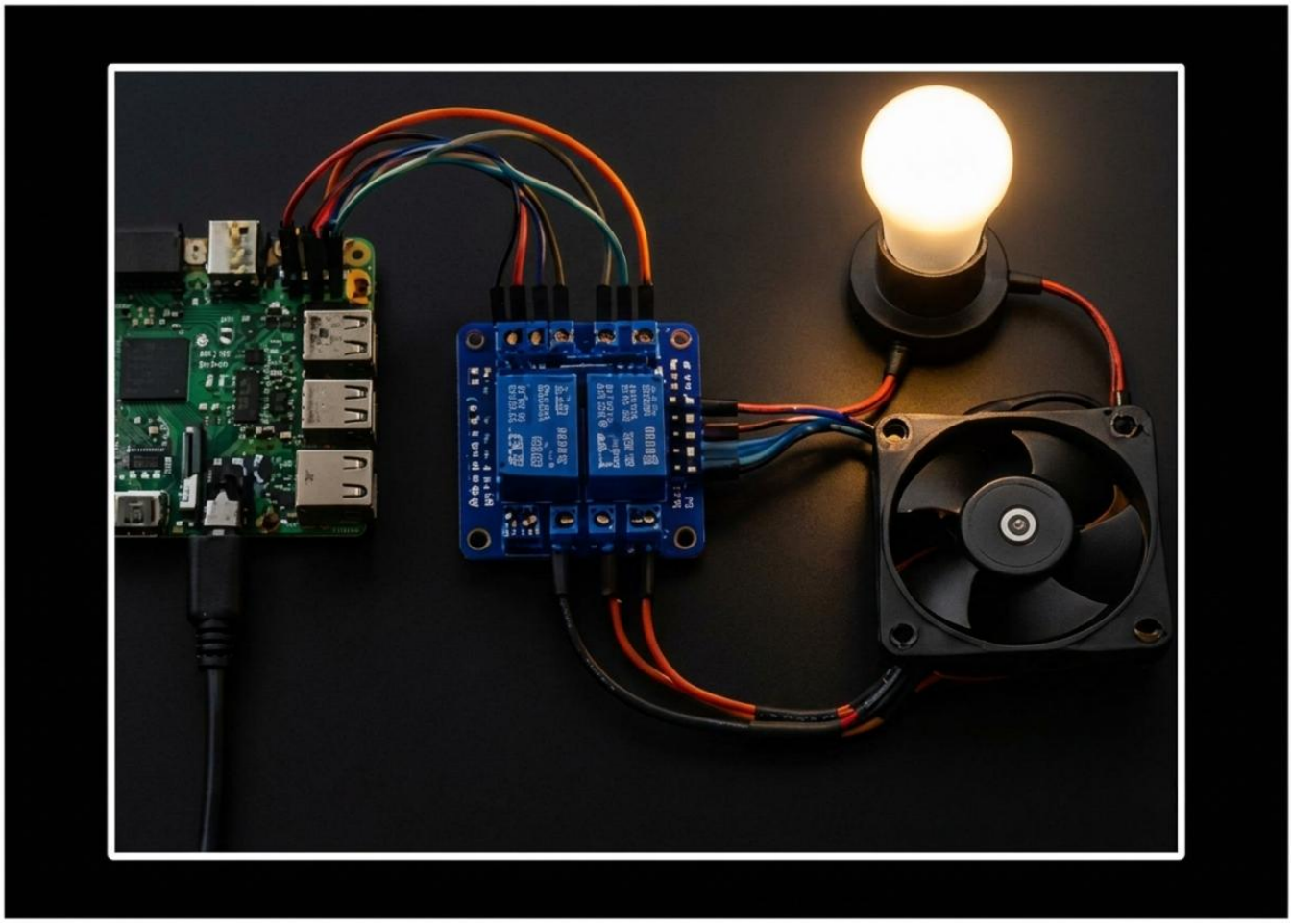
7. Execute the program

1. Run the program.
2. Find the Raspberry Pi's IP address: `hostname -I` (e.g. 192.168.1.10)
3. On a mobile phone or laptop on the same network, open a browser and navigate to `http://192.168.1.10:5000`

Result / Observation:

The web page loads with ON/OFF links for the Light and Fan; clicking a link switches the corresponding relay-controlled appliance accordingly.

8. Output



PRACTICAL – 11

1. Aim: Interface Fingerprint Sensor and 16x2 LCD with Raspberry Pi and display the message "Fingerprint Matched" or "Unmatched".

2. Prior Concepts

- Concept of Python Programming
- Basics of UART serial communication
- Basics of I2C communication
- Basic LCD display programming

3. New Concepts

- Interfacing a biometric fingerprint sensor with Raspberry Pi
- Using UART communication for fingerprint sensor data exchange
- Interfacing a 16x2 LCD through the I2C protocol
- Developing a complete fingerprint-based access verification system

4. Hardware Required

- Fingerprint Sensor (R307, AS608, etc.)
- 16x2 LCD with I2C module (PCF8574)
- Jumper wires

Pin Connection

Fingerprint Sensor	Raspberry Pi
TX	GPIO15 (RXD)
RX	GPIO14 (TXD)
VCC	5V
GND	GND

5. Procedure

1. Install Python3: `sudo apt install idle3`
2. Enable UART: `sudo raspi-config` → Interface Options → Serial Port → Login Shell? No, Serial Hardware? Yes.
3. Enable I2C: `sudo raspi-config` → Interface Options → I2C → Enable.
4. `sudo reboot`
5. `sudo apt update`
6. `pip3 install pyfingerprint`
7. `pip3 install RPLCD`

6. Code

```
from pyfingerprint.pyfingerprint import PyFingerprint
from RPLCD.i2c import CharLCD
from time import sleep

# LCD configuration
```

```
lcd = CharLCD('PCF8574', 0x27)
lcd.clear()
lcd.write_string('Place Finger')

try:
    f = PyFingerprint('/dev/serial0', 57600, 0xFFFFFFFF, 0x00000000)
    if not f.verifyPassword():
        raise ValueError('Fingerprint sensor password incorrect!')
except Exception as e:
    print('Sensor initialization failed:', e)
    exit(1)

while True:
    lcd.clear()
    lcd.write_string('Place Finger')

    while not f.readImage():
        pass

    f.convertImage(0x01)
    result = f.searchTemplate()
    positionNumber = result[0]

    lcd.clear()
    if positionNumber >= 0:
        lcd.write_string('Fingerprint')
        lcd.crlf()
        lcd.write_string('Matched')
        print("Fingerprint Matched")
    else:
        lcd.write_string('Fingerprint')
        lcd.crlf()
        lcd.write_string('Unmatched')
        print("Fingerprint Unmatched")

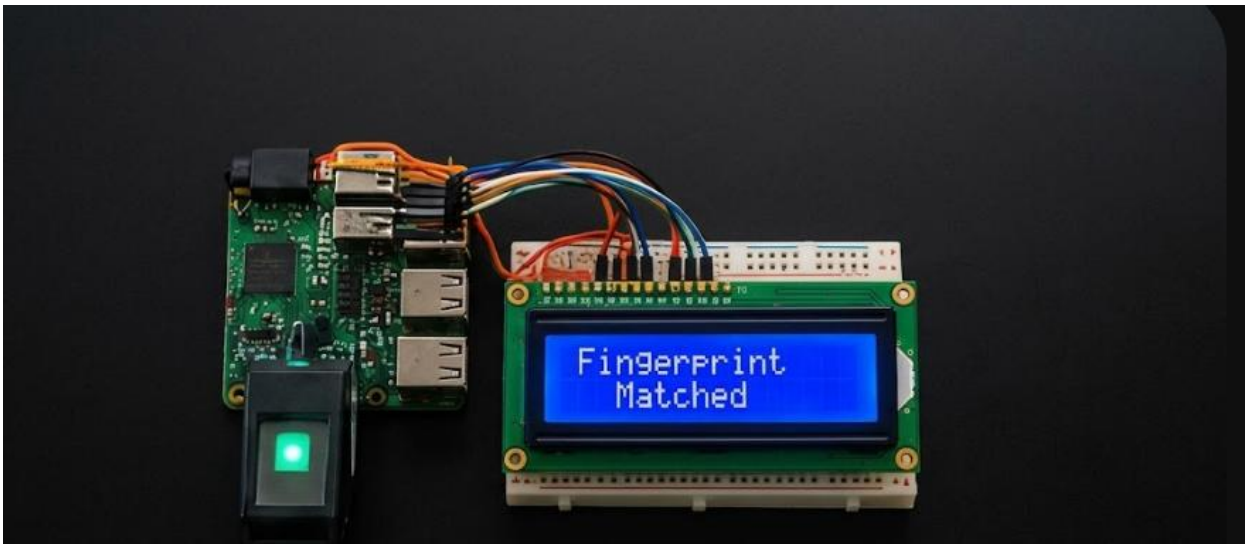
    sleep(3)
```

Result / Observation:

The LCD first prompts “Place Finger”; once a finger is scanned, it displays “Fingerprint Matched” if the print exists in the sensor's database, or “Fingerprint Unmatched” otherwise.

8. Output

9.



PRACTICAL – 12

1. Aim: Simulate a door lock by interfacing a fingerprint sensor and a relay module with Raspberry Pi. (A bulb is used instead of a door to show the output.)

2. Prior Concepts

- Concept of Python Programming
- UART communication
- Working of relay modules

3. New Concepts

- Using a fingerprint match to trigger a relay-driven actuator
- Simulating an electronic door-lock mechanism

4. Hardware Required

- Fingerprint sensor
- 1-channel relay module
- Bulb
- Jumper wires

Pin Connection

Component	Raspberry Pi Pin
Fingerprint TX	GPIO15 (RXD)
Fingerprint RX	GPIO14 (TXD)
Fingerprint VCC / GND	5V / GND
Relay IN	GPIO17 (Pin 11)
Relay VCC / GND	5V / GND

5. Procedure

1. Enable UART: `sudo raspi-config` → Interface Options → Serial Port.
2. `sudo reboot`
3. `pip3 install pyfingerprint RPi.GPIO`

6. Code

```
from pyfingerprint.pyfingerprint import PyFingerprint
import RPi.GPIO as GPIO
from time import sleep

RELAY = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY, GPIO.OUT)
GPIO.output(RELAY, GPIO.LOW)    # Door locked / bulb off

try:
    f = PyFingerprint('/dev/serial0', 57600, 0xFFFFFFFF, 0x00000000)
    if not f.verifyPassword():
        raise ValueError('Fingerprint sensor password incorrect!')
```

```
except Exception as e:
    print('Sensor init failed:', e)
    exit(1)

print("Simulated Door Lock System Ready")

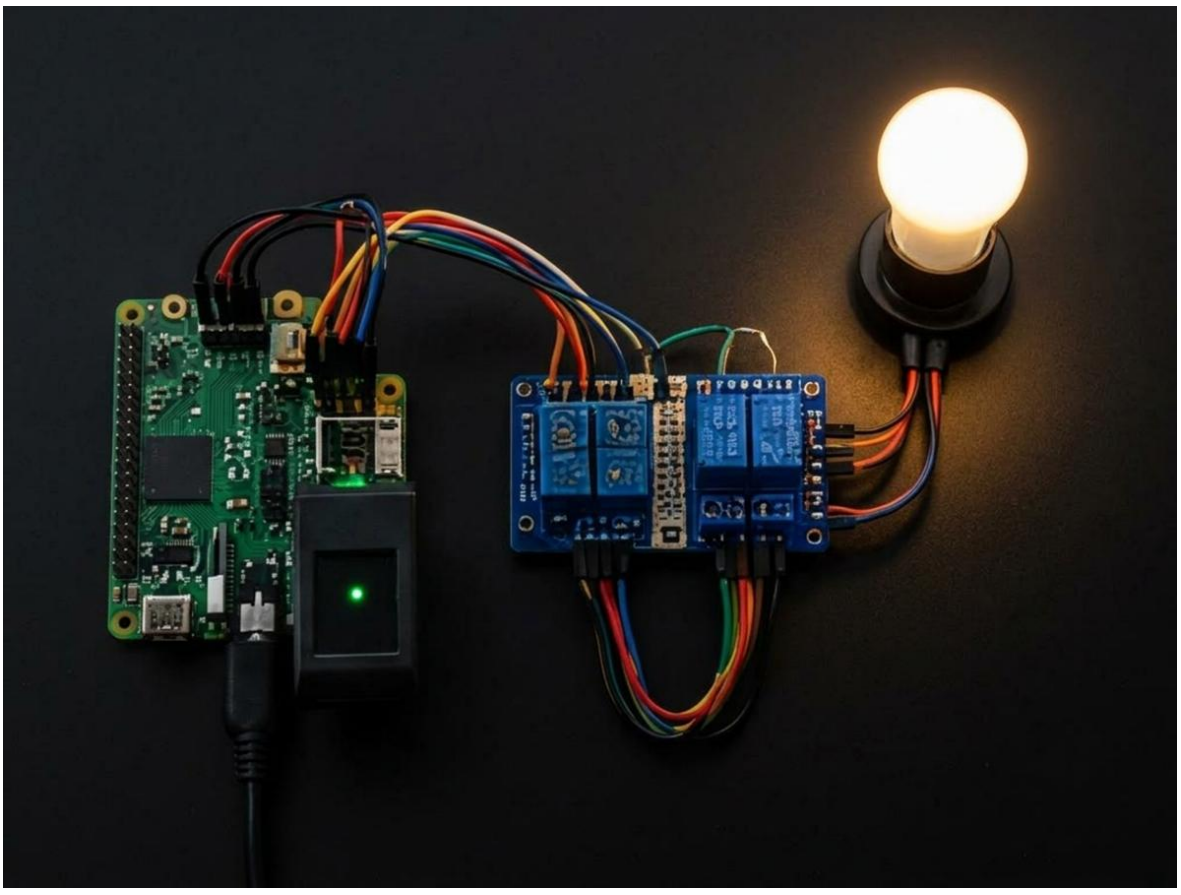
try:
    while True:
        print("Place finger to unlock...")
        while not f.readImage():
            pass
        f.convertImage(0x01)
        result = f.searchTemplate()
        positionNumber = result[0]

        if positionNumber >= 0:
            print("Access Granted - Door Unlocked (Bulb ON)")
            GPIO.output(RELAY, GPIO.HIGH)
            sleep(5)
            GPIO.output(RELAY, GPIO.LOW)
            print("Door Locked Again (Bulb OFF)")
        else:
            print("Access Denied")
        sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Result / Observation:

On a matched fingerprint the relay energizes and the bulb lights up for 5 seconds (simulating the door unlocking) and then switches off automatically (simulating re-locking); on an unmatched fingerprint, access is denied and the bulb stays off.

8. Output



PRACTICAL – 13

1. Aim: Implement a basic Publisher – Subscriber system using MQTT. Publish messages using paho-mqtt. Subscribe and control an LED.

2. Prior Concepts

- Concept of Python Programming
- Basic networking concepts
- Concept of publish-subscribe messaging

3. New Concepts

- MQTT protocol, brokers and topics
- The paho-mqtt client library
- Controlling GPIO from subscribed MQTT messages

4. Hardware Required

- LED
- Resistor
- Breadboard
- Jumper wires

5. Procedure

1 Install the MQTT client library: `pip3 install paho-mqtt`

2 Use a public broker (e.g. broker.hivemq.com) or run a local broker: `sudo apt install mosquitto mosquitto-clients -y`

6. Code – Publisher (publisher.py)

```
import paho.mqtt.client as mqtt

BROKER = "broker.hivemq.com"
TOPIC = "raspi/led"

client = mqtt.Client()
client.connect(BROKER, 1883, 60)

while True:
    msg = input("Enter command (on/off): ")
    client.publish(TOPIC, msg)
    print(f"Published: {msg}")
```

Code – Subscriber (subscriber.py, runs on the Raspberry Pi)

```
import paho.mqtt.client as mqtt
import RPi.GPIO as GPIO

LED = 17
BROKER = "broker.hivemq.com"
TOPIC = "raspi/led"

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)
GPIO.output(LED, GPIO.LOW)
```

```
def on_connect(client, userdata, flags, rc):
    print("Connected with result code " + str(rc))
    client.subscribe(TOPIC)


def on_message(client, userdata, msg):
    command = msg.payload.decode()
    print(f"Received: {command}")
    if command == "on":
        GPIO.output(LED, GPIO.HIGH)
    elif command == "off":
        GPIO.output(LED, GPIO.LOW)


client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, 1883, 60)
client.loop_forever()
```

Result / Observation:

Messages typed into the publisher (“on”/“off”) are relayed through the MQTT broker to the subscriber running on the Raspberry Pi, which switches the LED on or off accordingly – demonstrating the publish-subscribe pattern end to end.

8. Output

